

Building a Lean Cloud-Hosted Manus.im Clone: Technical Architecture & Survival Guide

Audience: Technical solo founder with a limited budget attempting to clone Manus.im.
Purpose: Ruthless, no-fluff architecture analysis. No motivational language. No generic startup advice. **Date:** March 2026

1. Executive Summary

Manus.im is an autonomous AI agent platform that allocates a dedicated cloud virtual machine (Ubuntu Linux) per user task, runs an iterative agent loop (Analyze → Plan → Execute → Observe), and uses executable Python code as its primary action mechanism — a pattern called “CodeAct” [1](#). The platform is built on top of Claude 3.5_{3.7} and fine-tuned Qwen models, with multi-model dynamic invocation for different subtask types [2](#).

Building a functional clone is technically feasible. The open-source ecosystem has matured enough that you do not need to build sandboxing, auth, billing, or observability from scratch. The real challenge is not capability — it is **operational survivability**: preventing infinite loops, controlling LLM API costs, securing code execution, and not going bankrupt before you hit 100 paying users.

The three decisions that determine whether this project lives or dies:

1. Whether you use a managed sandbox provider (E2B, Daytona) or build your own — building your own is a 3-month detour that will not be more secure.
2. Whether you implement step limits and cost quotas before launch — without them, a single malicious or confused user can drain your LLM budget overnight.
3. Whether you use a task queue from day one — running agent tasks synchronously on a web server will cause timeouts, lost state, and zombie

processes.

Realistic cost floor: 50–150/month at zero users (base infrastructure). 500–1,500/month at 100 active users, dominated by LLM API costs.

2. Manus Architecture: What You Are Actually Cloning

Before building, understand what Manus actually is under the hood.

2.1 Foundation Model Backbone

Manus does not run a proprietary model. It uses Claude 3.5₃.7 Sonnet as the primary reasoning engine, supplemented by fine-tuned Qwen models for specific subtasks [2](#). It routes different subtask types to different models dynamically. This is the “multi-model dynamic invocation” pattern — using Claude for complex logical reasoning, cheaper models for formatting and extraction.

Implication: You do not need to train anything. You need a model router.

2.2 The Sandbox (Cloud VM per Task)

Each Manus task gets a dedicated Ubuntu Linux VM with full internet access, a shell, Python/Node.js interpreters, a file system, and a browser [1](#). The VM sleeps when idle (7 days for free users, 21 days for Pro) and is recycled if inactive beyond that window. Key files (artifacts, uploads) are restored on recreation; intermediate files are not.

Implication: You need per-task isolated execution environments. The cheapest managed option is E2B (~0.08/hour per sandbox) or Daytona (0.08/hour, sub-90ms cold starts) [3](#).

2.3 The Agent Loop

Manus operates on a strict iterative loop:

1. Analyze current state (**event** stream)
2. Plan **next** action (Planner **module**)
3. Execute one tool **call** (CodeAct – write Python, run it)
4. Observe result (append **to event** stream)
5. Repeat **until** task complete **or step** limit hit

One tool call per iteration. The agent cannot batch actions. This is intentional — it allows the system to monitor each step and prevents runaway execution [2](#).

Implication: Your orchestrator must enforce a maximum iteration count. Without it, a confused agent will loop until your API budget is gone.

2.4 Memory Architecture

Manus uses two memory layers:

- **Short-term:** The event stream (messages array passed to the LLM each iteration). Grows with each tool call. Must be managed explicitly to avoid context window overflow.
- **Long-term:** File-based memory. The agent writes `todo.md` and intermediate result files to the sandbox filesystem. These persist across the task lifecycle.

Implication: You do not need a vector database for MVP. A Postgres table for conversation history and a file store for artifacts is sufficient.

2.5 Multi-Agent Coordination

Manus uses specialized sub-agents for parallel work (research agent, coding agent, review agent), coordinated by a high-level orchestrator [2](#). Each sub-agent runs in its own sandbox.

Implication: Do not build this first. A single agent with good tools is more reliable, cheaper to debug, and sufficient for 90% of use cases. Multi-agent adds coordination overhead, new failure modes, and inter-agent serialization contracts you do not want to maintain as a solo founder.

3. Minimum Viable Cloud Architecture

The MVP must handle: user auth, task submission, async agent execution, sandboxed code running, file storage, and billing. Everything else is optional at launch.

3.1 Component Stack

Layer	Component	Provider	Monthly Cost (Baseline)
Frontend	Next.js chat UI + task dashboard	Vercel (free tier)	\$0
Auth	User sessions, API keys	Supabase Auth	\$0 (free tier)
API Server	REST/WebSocket for task submission	Render Web Service	\$7
Task Queue	Async job dispatch, retries, timeouts	BullMQ + Upstash Redis	\$10
Orchestrator Worker	Agent loop execution	Render Background Worker	\$7
Sandbox	Isolated code execution per task	E2B	~\$0.08/hr per sandbox
Database	Task state, conversation history	Supabase PostgreSQL	\$0 (free tier)
File Storage	User uploads, agent artifacts	Supabase Storage	\$0 (free tier)
LLM Gateway	Model routing, cost tracking, caching	Helicone	\$0 (free tier)
Observability	Error tracking, logs	Sentry (free) + Langfuse (free)	\$0
Billing	Credits, usage metering	Stripe	2.9% + \$0.30/transaction

Total baseline: \$24/month before any LLM API costs or sandbox usage.

3.2 LLM Cost Reality

LLM API costs are the primary variable cost and the biggest financial risk. A single complex agent task can consume 50,000–200,000 tokens across all steps (planning, tool calls, observations, retries).

Model	Input (per 1M tokens)	Output (per 1M tokens)	Best Use in Agent
Claude 3.7 Sonnet	\$3.00	\$15.00	Complex reasoning, planning
GPT-4o	\$2.50	\$10.00	General tool calling
GPT-4o-mini	\$0.15	\$0.60	Formatting, extraction, retries
Gemini 2.5 Flash	\$0.10	\$0.40	High-volume cheap steps
DeepSeek V3	\$0.14	\$0.28	Cost-optimized reasoning

At 100,000 tokens per task (a moderate research task), using Claude 3.7 Sonnet exclusively costs ~1.80*per task*. *Routing* 600.50–\$0.80 per task. Model routing can cut per-task LLM costs by 40–60% with minimal accuracy impact [4](#).

4. What NOT to Build First

These are the most common time and money traps for solo founders building agent platforms.

4.1 Custom Docker Sandboxing

The trap: Building your own secure, isolated code execution environment using Docker, gVisor, or Firecracker.

Why it’s a trap: Security hardening a code execution environment is a 2–3 month engineering project. You need to handle network isolation, filesystem restrictions, resource limits, privilege escalation prevention, and process cleanup. E2B and Daytona have already solved this. E2B is open-source and can be self-hosted if needed [3](#).

What to do instead: Use E2B for MVP. Self-host it later if costs become significant.

4.2 Multi-Agent Orchestration

The trap: Building CrewAI/AutoGen-style agent swarms with specialized sub-agents from day one.

Why it's a trap: Every agent boundary is a new failure mode. Every inter-agent message is a serialization contract. Coordination overhead is significant. The orchestrator itself must be robust. You will spend more time debugging agent-to-agent communication than building features users care about [4](#).

What to do instead: Build a single agent with a comprehensive tool set. Add multi-agent only when you hit a real context length ceiling or genuine parallelism requirement.

4.3 Custom Auth and Billing

The trap: Rolling your own authentication system or building a custom credits/billing engine.

Why it's a trap: Auth vulnerabilities in agent platforms are particularly dangerous because the agent has broad permissions. A session hijacking bug in a standard SaaS is bad. In an agent platform where the agent can execute code and browse the web on behalf of the user, it is catastrophic.

What to do instead: Supabase Auth for user sessions and API keys. Stripe for billing with their native Credits feature (launched February 2025). Both are free at small scale.

4.4 Proprietary Model Training or Fine-Tuning

The trap: Fine-tuning a model on agent trajectories to improve performance.

Why it's a trap: You do not have enough data, enough compute budget, or enough time. Claude 3.7 Sonnet and GPT-4o are already excellent at tool calling and code generation. Fine-tuning adds maintenance overhead and model version management complexity.

What to do instead: Invest in prompt engineering and tool design. A well-designed tool with a clear description outperforms a fine-tuned model with a vague tool

interface.

4.5 Multi-Region Redundancy and High Availability

The trap: Building for 99.99% uptime before you have 10 paying users.

Why it's a trap: You will spend weeks on infrastructure that provides zero user value at your current scale. Render and Fly.io already provide reasonable uptime. Your bottleneck is product-market fit, not infrastructure reliability.

What to do instead: Single-region deployment. Implement graceful degradation (return partial results on failure) instead of redundancy.

5. Cloud Deployment Options: Deep Comparison

5.1 Serverless Functions (Lambda, Cloud Run, Vercel Functions)

The problem: Agent tasks run for 5–30 minutes. AWS Lambda has a 15-minute hard timeout. Vercel Functions have a 5-minute timeout on Pro. Cloud Run can be configured for longer timeouts but requires careful session management.

When it works: API endpoints (task submission, status polling), webhook handlers, and short utility functions. Not suitable for the agent execution loop itself.

Cost at scale: Serverless is cheap at low volume but expensive at high concurrency. AWS Lambda at 1,000 concurrent 10-minute tasks costs $\sim 0.0000166667 / GB - second \times 600seconds \times 1GB \times 1,000tasks = 10$ per batch. This is competitive, but cold starts (100ms–3s) add latency to every task start.

5.2 Container Apps / PaaS Workers (Render, Fly.io, Railway)

The sweet spot for agent workers. These platforms run long-lived processes with no timeout constraints, predictable pricing, and simple deployment.

Provider	Worker Pricing	Cold Start	Best For
Render	\$7/month (512MB RAM)	~30s (free tier), instant (paid)	Simple, predictable billing
Fly.io	~\$1.94/month (256MB) + usage	Fast	Global distribution, fine-grained control
Railway	Usage-based (\$0.000463/vCPU/min)	Instant	Fast iteration, variable workloads

Recommendation: Render for the orchestrator API (fixed \$7/month). Fly.io for worker nodes if you need global distribution. Railway if your workload is highly variable and you want pure usage-based billing.

5.3 Long-Running VPS (Hetzner, DigitalOcean, EC2)

For high-volume, cost-sensitive workloads. A Hetzner CX22 (2 vCPU, 4GB RAM) costs €4.35/month (~4.70). *AW Equivalent(t3.medium) costs 30/month* — a 6x premium [5](#).

The tradeoff: VPS requires you to manage deployments, process supervision (PM2, systemd), and scaling manually. At early stage, this operational overhead is not worth the cost savings. At 1,000+ active users, it becomes worth it.

5.4 Managed Sandbox Providers (E2B, Daytona, Modal)

These are the critical infrastructure layer for code execution and browser automation.

Provider	Cold Start	Max Runtime	Self-Host	Cost/Hour	Best For
E2B	150ms	24 hours (Pro)	Yes (OSS)	\$0.08	Quick integration, great SDKs
Daytona	90ms	Unlimited	Enterprise	\$0.08	Full dev environments, stateful
Modal	Sub-second	24 hours	No	\$0.12	Python ML workloads, GPU access
Blaxel	25ms	Unlimited	No	\$0.08	Ultra-fast resume, stateful agents

Recommendation: E2B for MVP. It has the best SDK, is open-source (can self-host to reduce costs later), and has a \$100 free credit on signup [3](#).

5.5 Browser Automation Pools

Running headless Chromium at scale is expensive and operationally complex. Bot detection (Cloudflare Turnstile, Datadome, PerimeterX) blocks standard Playwright/Puppeteer.

Provider	Concurrent Browsers	Monthly Cost	Bot Detection Bypass
Browserless.io	10 (Prototyping)	\$25	Partial
Browserless.io	40 (Starter)	\$140	Partial
Scrapfly	Variable	\$9+	Yes (proxy rotation)
BrowserCloud	5 (Hobby)	\$19	Partial
Self-hosted Playwright	Unlimited	VPS cost only	None

Recommendation: Self-hosted Playwright inside E2B sandboxes for MVP (each sandbox gets its own browser, no pool management needed). Upgrade to Browserless or Scrapfly only when bot detection becomes a real problem.

6. Required Subsystems: Minimum Viable Implementation

6.1 Frontend

What you need: A chat interface with real-time task status updates, a file upload/download panel, and a task history view.

Stack: Next.js + Tailwind + shadcn/ui. Deploy on Vercel free tier. Use WebSockets or Server-Sent Events (SSE) for real-time agent step streaming.

What you do NOT need at MVP: A custom design system, mobile app, collaborative editing, or a complex dashboard. Ship the chat UI first.

6.2 API Gateway

What you need: Request routing, auth middleware (validate Supabase JWT), rate limiting, and request logging.

Stack: Cloudflare Workers as a lightweight API gateway. Alternatively, implement rate limiting directly in your Render API server using a Redis-backed token bucket.

Rate limiting strategy: Token-based limiting (e.g., 100,000 tokens/hour per user) rather than request-based [6](#). This maps directly to your cost model and prevents a single user from running 50 concurrent agent tasks.

6.3 Worker Queue

What you need: Async task dispatch, job status tracking, retries with exponential backoff, and dead-letter queues for failed jobs.

Stack: BullMQ (Node.js) or Celery (Python) with Redis as the broker. For a managed option, Upstash Redis costs *~0/month on the free tier (10,000 commands/day)* and 10/month on the pay-as-you-go tier.

Critical configuration:

- Set a `jobTimeout` of 600,000ms (10 minutes) as the hard wall-clock limit.
- Set `attempts: 3` with exponential backoff for transient failures.

- Set `removeOnComplete: 100` and `removeOnFail: 500` to prevent Redis memory bloat.
- Implement a dead-letter queue for jobs that fail all retry attempts.

6.4 Sandbox / Runner

What you need: An isolated execution environment per task that can run Python code, shell commands, and a browser.

Stack: E2B SDK. Create a sandbox at task start, execute agent-generated code inside it, and close it when the task completes. Each sandbox is a Linux microVM with a dedicated filesystem.

Critical security note: Never run agent-generated code on your main application server or in a shared container. The agent will eventually generate code that attempts to read environment variables, access the filesystem outside its scope, or make unexpected network calls. E2B's microVM isolation prevents this at the hypervisor level [1](#).

6.5 Memory and State Store

Short-term memory (conversation context): Store the full message history in Postgres (Supabase). Implement a sliding window: keep the last 20 messages in the LLM context, summarize older messages into a single “history summary” message. This prevents context window overflow without losing task continuity.

Long-term memory: Supabase pgvector for semantic search over past task outputs and user preferences. Do not build a complex memory system at MVP — a simple vector store with cosine similarity search is sufficient.

Task state: Store the full agent state (current step, tool outputs, plan) as a JSON column in Postgres. This enables checkpointing: if a worker crashes, the task can resume from the last persisted state [4](#).

6.6 Auth

Stack: Supabase Auth. Provides email/password, OAuth (Google, GitHub), and API key management out of the box. At 100,000 MAU, Supabase Auth costs ~25/month versus Auth0 at 500+/month [7](#).

Critical for agent platforms: Implement API key scoping. Each API key should have explicit permissions (e.g., `task:create`, `task:read`, `file:upload`). An API key with full permissions that gets leaked gives an attacker full agent execution capabilities.

6.7 Observability

What you need: LLM call tracing (prompts, completions, token counts, costs), task execution traces (which tools were called, in what order, with what results), error tracking, and basic infrastructure metrics.

Stack:

- **Langfuse** (open-source, self-hostable): LLM traces, cost tracking, prompt management. Free cloud tier available.
- **Sentry:** Error tracking and performance monitoring. Free tier covers 5,000 errors/month.
- **Uptime monitoring:** Better Uptime or UptimeRobot (free tier).

What you do NOT need: Datadog (\$\$\$), full OpenTelemetry pipeline, custom Grafana dashboards. These are premature at sub-1,000 users.

The observability gap that kills agent platforms: An agent that is stuck in a loop will appear “healthy” by standard infrastructure metrics (process is running, no exceptions thrown). You need LLM-level tracing to detect when an agent is making the same tool call repeatedly with no progress [8](#).

6.8 Model Routing

The pattern: Route different agent steps to different models based on complexity. Do not use your most expensive model for every step.

Implementation with LiteLLM or Helicone:

- Planning and decomposition steps → Claude 3.7 Sonnet or GPT-4o
- Tool call execution (formatting, extraction) → GPT-4o-mini or Gemini 2.5 Flash
- Retries after failure → GPT-4o-mini (cheaper, acceptable quality for retry logic)
- Simple classification → Gemini 2.5 Flash Lite (\$0.10/1M input tokens)

Cost impact: A hybrid routing strategy can reduce average per-task LLM costs by 40–60% with minimal accuracy impact [4](#).

6.9 Billing and Credits

Architecture: Pre-paid credits system. Users buy credits upfront (e.g., \$10 = 1,000 credits). Each agent task consumes credits based on actual usage (tokens + sandbox minutes). This protects you from running at a loss on complex tasks.

Implementation: Stripe Billing with their native Credits feature (launched February 2025). Stripe handles credit purchase, balance tracking, and overage handling. Decrement credits in your database after each LLM call and sandbox minute consumed.

Critical: Set a minimum credit balance check before starting any task. If a user has zero credits, reject the task immediately. Do not run tasks on credit.

6.10 Abuse Prevention

Threat model: Users who try to use your platform as a free LLM API, run cryptocurrency miners in sandboxes, or perform web scraping at scale.

Controls:

- Hard credit limits: No task starts without sufficient credits.
 - Sandbox network restrictions: Block outbound connections to known mining pools and suspicious IP ranges.
 - Token quotas per task: Hard cap at 200,000 tokens per task run regardless of credits.
 - Rate limiting at the API gateway: Maximum 5 concurrent tasks per user.
 - Anomaly detection: Alert if a single user consumes more than 10x the average daily credit usage.
-

7. Failure Mode Analysis

These are the failure modes that will kill your platform first, in rough order of likelihood.

7.1 The Infinite Loop (Highest Probability)

How it manifests: The agent hits an error (e.g., a tool returns an unexpected schema), generates a slightly different tool call, gets the same error, and repeats. The task runs until it hits the context window limit or your API budget is exhausted. The process appears healthy. No exceptions are thrown.

How bad it gets: A single looping task on Claude 3.7 Sonnet can consume 5–20 in API costs before hitting the context window limit. At 100 users, even a 1% loop rate means 1 looping task per 100 tasks — manageable. At 10,000 tasks/day, that is 100 looping tasks per day at 5–20 each = 500–2,000/day in wasted API costs.

Prevention:

- Hard step limit: `max_iterations = 20`. When hit, return the best partial result with a clear failure signal.
- Failure taxonomy: Distinguish transient failures (retry) from structural failures (abort). A tool returning an invalid schema is a structural failure — do not retry it 3 times.
- LLM-level tracing: Detect when the same tool is called with the same parameters 3 times in a row and abort.

7.2 Cost Explosion via Retry Amplification

How it manifests: A task with a 20% retry rate does not cost 20% more. It costs 20% more per retry, compounded across all retried steps. A task that retries 3 times before succeeding costs 3x the token spend plus 3x the latency⁴.

How bad it gets: If your average task costs 0.50 in LLM tokens and you have a $300.50 \times (1 + 0.3 \times 2) = 0.80$. At 1,000 tasks/day, that is 240/day versus \$150/day — a 60% cost overrun that compounds monthly.

Prevention:

- Route retries to cheaper models (GPT-4o-mini instead of Claude 3.7 Sonnet).
- Implement circuit breakers: If a tool fails 3 times in a row, stop calling it and return a partial result.
- Per-task token budget: Hard cap at 150,000 tokens per task. If the budget is exceeded, abort and return partial results.

7.3 The Subagent Black Hole

How it manifests: A spawned subtask fails silently in its isolated session. The parent task waits indefinitely for a completion signal that never arrives. The parent task appears to be running normally [9](#).

How bad it gets: The parent task occupies a worker slot indefinitely, blocking other tasks from executing. If you have 5 worker slots and 3 are occupied by zombie parent tasks, your effective throughput drops by 60%.

Prevention:

- Wall-clock timeout on every external call and subagent spawn. Default: 60 seconds. Tasks that need longer must opt in explicitly.
- Output verification: Do not wait for a completion signal. Verify that the expected output file exists and contains valid data before marking a subtask complete.
- Independent watchdog process: A separate process that monitors task health, not just process existence.

7.4 Context Window Overflow

How it manifests: The message history grows with each tool call. After 20–30 steps, the full context exceeds the model’s context window. The model silently drops early messages — including the original task goal and constraints. The agent starts making decisions that contradict the user’s original request.

How bad it gets: The task appears to be running normally. The agent produces output, but it is wrong in ways that are not immediately obvious. Users get confused results and lose trust in the platform.

Prevention:

- Explicit context management: Keep only the last N tool outputs in the active context. Summarize older steps into a single “history summary” message.
- Context length monitoring: Track token count at each step. If it exceeds 80% of the model’s context window, trigger summarization before the next step.
- File-based memory: Write intermediate results to the sandbox filesystem (like Manus does with `todo.md`). The agent reads these files instead of relying on the message history.

7.5 Browser Bot Detection

How it manifests: Playwright/Puppeteer requests are blocked by Cloudflare Turnstile, Datadome, or PerimeterX. The browser automation tool returns empty pages or CAPTCHA challenges. The agent retries repeatedly, consuming tokens and sandbox time without making progress.

How bad it gets: Any task that requires browsing popular websites (LinkedIn, Amazon, news sites) will fail silently or get stuck in a retry loop. This is a fundamental limitation of standard headless browsers.

Prevention:

- Use Browserless.io or Scrapfly for sites with known bot detection. They handle proxy rotation and fingerprint spoofing.
- Implement a fallback: If browser navigation fails 3 times, fall back to a search API (Serper, Tavily) for information retrieval.
- Do not promise browser automation on sites with aggressive bot detection (LinkedIn, Google, Amazon) in your product marketing.

7.6 Task Queue Failures

How it manifests: Redis goes down, causing all queued jobs to be lost. Or a worker crashes mid-task, leaving the task in a “processing” state indefinitely (a “stalled job”). Or a malformed task payload causes a worker to crash on every attempt, blocking the queue (a “poison pill” job).

Prevention:

- Use BullMQ's built-in stall detection: Jobs that do not send a heartbeat within `stalledInterval` are automatically re-queued.
- Implement a dead-letter queue: Jobs that fail all retry attempts are moved to a separate queue for manual inspection.
- Validate task payloads at submission time: Reject malformed payloads before they enter the queue.
- Use Upstash Redis (managed, persistent) rather than a self-hosted Redis instance that you might forget to back up.

7.7 Storage Cost Explosion

How it manifests: Agent tasks generate large artifacts (PDFs, code files, screenshots, data exports). These accumulate in your file storage without any cleanup policy. After 6 months, you are paying 50–200/month for storage of files that no user has accessed in months.

Prevention:

- Implement a retention policy: Delete sandbox artifacts older than 30 days for free users, 90 days for paid users (similar to Manus's approach [1](#)).
- Store only final artifacts (user-facing outputs) in persistent storage. Intermediate files (temporary code, partial results) should live in the ephemeral sandbox filesystem and be discarded when the sandbox closes.
- Set storage quotas per user: Free users get 1GB, paid users get 10GB.

7.8 Auth Vulnerabilities Specific to Agent Platforms

How it manifests: A session token is leaked or an API key is compromised. In a standard SaaS, this means the attacker can read data. In an agent platform, this means the attacker can execute arbitrary code in a sandbox, browse the web as the user, and potentially access the user's connected services (OAuth tokens stored in the sandbox).

Prevention:

- Short-lived session tokens (1 hour, not 30 days).
- API key scoping: Each key has explicit permissions. No "full access" keys.

- Sandbox isolation: Even if an attacker gains access to one user's sandbox, they cannot access other users' sandboxes or the host system [1](#).
 - Audit logging: Log every agent action (tool calls, file operations) with the user ID and session token. This enables forensic analysis after a breach.
-

8. Prioritized Fix List for a Rough Current Setup

Assuming your current state: basic chat UI, some LLM API calls work, maybe a basic task runner, but no proper queue, no sandboxing, no proper error handling, no billing, no observability, and browser automation is fragile.

Execute these fixes in strict order. Each one unblocks the next.

Fix #1: Hard Wall-Clock Timeouts on Every Task (Day 1)

Why first: Without timeouts, a single stuck task can occupy a worker indefinitely, block your queue, and drain your API budget. This is the fastest path to a production incident.

Implementation: In BullMQ, set `jobTimeout: 600000` (10 minutes). In your agent loop, set `max_iterations: 20`. Kill any task that exceeds either limit and return a partial result.

Fix #2: Move Agent Execution to a Proper Task Queue (Days 1–3)

Why second: Running agent tasks synchronously on your web server will cause HTTP timeouts (most platforms kill connections after 30–60 seconds), lost state on server restart, and no retry capability.

Implementation: BullMQ + Upstash Redis. Submit tasks to the queue and return a job ID immediately. Poll for status via a `/tasks/{id}/status` endpoint or stream updates via SSE.

Fix #3: Move Code Execution to E2B Sandboxes (Days 3–7)

Why third: Running agent-generated code on your own server is a critical security vulnerability. The agent will eventually generate code that reads your environment

variables, accesses your database credentials, or makes unexpected network calls.

Implementation: Install the E2B SDK. Create a sandbox at task start, execute all agent-generated code inside it, and close the sandbox when the task completes. Map the sandbox filesystem to Supabase Storage for artifact persistence.

Fix #4: Integrate an LLM Gateway for Cost Visibility (Days 7–10)

Why fourth: You cannot fix what you cannot measure. Without per-user, per-task cost tracking, you will not know which tasks are expensive, which users are abusing the system, or whether your billing covers your costs.

Implementation: Route all LLM calls through Helicone or LiteLLM. Both provide per-request cost tracking, caching for identical requests, and model fallback configuration. Helicone's free tier covers 10,000 requests/month.

Fix #5: Implement Credit-Based Billing with Pre-Task Balance Check (Days 10–14)

Why fifth: Every task you run without billing is a loss. Even if you are in beta, implement a credit system now. It is much harder to add billing to an existing user base than to launch with it.

Implementation: Stripe Credits + Supabase for balance tracking. Check credit balance before starting any task. Reject tasks from users with zero credits. Decrement credits after each LLM call and sandbox minute consumed.

Fix #6: Add State Checkpointing to the Agent Loop (Days 14–21)

Why sixth: Without checkpointing, a worker crash mid-task means the user loses all progress and must restart from scratch. This destroys trust.

Implementation: After each tool call, persist the full agent state (current step, tool outputs, plan, event stream) as a JSON column in Postgres. If a worker crashes, the task is re-queued and resumes from the last persisted state. LangGraph handles this natively via its checkpointing model.

Fix #7: Implement Step-Level Model Routing (Days 21–30)

Why seventh: Once you have cost visibility (Fix #4), you will see which steps are expensive. Route routine steps (formatting, extraction, simple classification) to GPT-4o-mini or Gemini 2.5 Flash. Reserve Claude 3.7 Sonnet for planning and complex reasoning.

Expected impact: 40–60% reduction in per-task LLM costs with minimal accuracy impact.

9. v1 Roadmap (First 90 Days)

Week	Milestone	Key Deliverable
1–2	Core infrastructure	Supabase, BullMQ, Render workers, basic auth
3–4	Agent loop	LangGraph orchestrator, E2B sandbox integration, 5 core tools
5–6	Async task system	Task submission, status polling, SSE streaming
7–8	Billing	Stripe Credits, pre-task balance check, usage tracking
9–10	Observability	Helicone integration, Langfuse tracing, Sentry errors
11–12	Hardening	Step limits, timeouts, circuit breakers, abuse prevention

10. Anti-Roadmap (Explicitly Do NOT Build in v1)

Feature	Why Not
Custom Docker sandboxing	3-month engineering detour. Use E2B.
Multi-agent orchestration	Adds coordination failure modes. Single agent first.
Custom auth system	Auth vulnerabilities in agent platforms are catastrophic. Use Supabase.
Proprietary model fine-tuning	No data, no budget, no time. Use Claude/GPT-4o APIs.
Multi-region deployment	Premature. Single-region is fine for first 1,000 users.
Real-time collaboration	Complex state management. Not a v1 feature.
Mobile app	Web-first. Mobile adds platform complexity without revenue.
Custom vector database	pgvector in Supabase is sufficient for MVP RAG.
Slide generation	Low-leverage feature. Build after core agent loop is stable.
MCP server ecosystem	Integration complexity. Add after core tools work reliably.

11. Cost-Risk Analysis Table

Scale	Monthly Cost Range	Primary Cost Driver	Biggest Financial Risk
0 users (dev)	20–50	Base hosting (Render + Supabase)	Over-engineering infrastructure before launch
10 active users	100–250	LLM API costs	Uncapped token usage per task
100 active users	500–1,500	LLM APIs + E2B sandbox compute	Malicious users draining credits before billing check
500 active users	2,000–5,000	LLM APIs + browser automation	Retry amplification from buggy tool implementations
1,000 active users	4,000–10,000	LLM APIs + compute + storage	Database write contention, queue throughput limits

Cost categories by growth rate (fastest to slowest):

1. LLM API costs (grows linearly with task volume, accelerates with retry amplification)
2. Sandbox compute (grows linearly with concurrent tasks)
3. Browser automation (grows with tasks requiring web browsing)
4. Storage (grows slowly but compounds without a retention policy)
5. Base infrastructure (nearly fixed, grows in steps as you scale worker count)

Break-even analysis: At 0.80 average LLM cost per task and 0.10 sandbox cost per task, your minimum viable price per task is ~1.50 (including 50¢ per 100 credits, you break even at ~100 tasks/month per paying user).

12. “Fix This First” Checklist

Execute in order. Do not skip steps.

- ☐ **Set hard wall-clock timeouts on all tasks** (10-minute max, kill and return partial result)
 - ☐ **Add step limits to the agent loop** (max 20 iterations, abort with partial result when hit)
 - ☐ **Move all agent execution to a BullMQ worker queue** (decouple from web server, enable retries)
 - ☐ **Move code execution to E2B sandboxes** (stop running agent code on your server — it is a security hole)
 - ☐ **Route all LLM calls through Helicone or LiteLLM** (get cost visibility before billing launch)
 - ☐ **Implement credit-based billing with pre-task balance check** (reject zero-credit tasks immediately)
 - ☐ **Add state checkpointing after every tool call** (enable task resume after worker crash)
 - ☐ **Implement step-level model routing** (GPT-4o-mini for routine steps, Claude for planning)
 - ☐ **Add token quota per task** (hard cap at 150,000 tokens per run, abort and return partial result)
 - ☐ **Implement stall detection in the queue** (detect and kill zombie tasks that are not making progress)
 - ☐ **Add context window management** (summarize old steps, keep only last N tool outputs in active context)
 - ☐ **Set storage retention policies** (delete artifacts older than 30 days for free users)
-

References

1. Manus.im. “Understanding Manus Sandbox — Your Cloud Computer.” January 14, 2026. <https://manus.im/blog/manus-sandbox>
2. Renschni. “In-depth technical investigation into the Manus AI agent.” GitHub Gist, March 2026. <https://gist.github.com/renschni/4fbc70b31bad8dd57f3370239dccd58f>

3. Superagent. "AI Code Sandbox Benchmark 2026: Modal vs E2B vs Daytona vs Cloudflare vs Vercel vs Beam vs Blaxel." January 2026.
<https://www.superagent.sh/blog/ai-code-sandbox-benchmark-2026>
4. Ranjan Kumar. "Designing Agentic AI Systems That Survive Production." March 5, 2026. <https://ranjankumar.in/designing-agentic-ai-ystems-that-survive-production>
5. DanubeData. "AWS vs Hetzner vs DanubeData: Real Cloud Cost Comparison 2025." December 2025. <https://danubedata.ro/blog/aws-vs-hetzner-vs-danubedata-cloud-cost-comparison-2025>
6. Zuplo. "Token-Based Rate Limiting: How to Manage AI Agent API Traffic in Production." March 8, 2026. <https://zuplo.com/learning-center/token-based-rate-limiting-ai-agents/>
7. AppStackBuilder. "Clerk vs Auth0 vs Supabase Auth 2026: Complete Comparison." 2026. <https://appstackbuilder.com/blog/clerk-vs-auth0-vs-supabase-auth>
8. OneUptime. "Monitoring AI Agents in Production: The Observability Gap." March 14, 2026. <https://oneuptime.com/blog/post/2026-03-14-monitoring-ai-agents-in-production/view>
9. Bob Renze. "How AI Agents Handle Stalled Tasks and Timeouts: Lessons From My Production Failure." DEV Community, March 4, 2026.
<https://dev.to/bobrenze/how-ai-agents-handle-stalled-tasks-and-timeouts-lessons-from-my-production-failure-1jj9>